

Data Transmission

Syllabus

2.1 Types & Methods of Data Transmission

- 1
 - (a) Understand that data is broken down into packets to be transmitted
 - (b) Describe the structure of a packet
 - A packet of data in a unit of data contains a:
 - Packet header
 - Payload
 - Trailer
 - The packet header includes the:
 - Destination address
 - Packet number
 - Originator's address
 - Data is broken down into packets
 - Each packet could take a different route
 - A router controls the route a packet takes
 - Packets may arrive out of order
 - Once the last packet has arrived, packets are reordered
 - (c) Describe the process of packet switching
- 2
 - (a) Describe how data is transmitted from one device to another using different methods of data transmission
 - Including:
 - Serial
 - Parallel
 - Simplex
 - Half-duplex
 - Full-duplex
 - (b) Explain the suitability of each method of data transmission, for a given scenario
 - Including the advantages and disadvantages of each method
- 3 Understand the universal serial bus (USB) interface and explain how it is used to transmit data
 - Including the benefits and drawbacks of the interface

2.2 Methods of Error Detection

- 1 Understand the need to check for errors after data transmission and how these errors can occur
 - Errors can occur during data transmission due to interference, e.g., data loss, data gain and data change.



+92-335-8467840



cswithsk@gmail.com

- 2 Describe the processes involved in each of the following error detection methods for detecting errors in data after transmission: parity check (odd and even), checksum and echo check
 - 3 Describe how a check digit is used to detect errors in data entry and identify examples of when a check digit is used, including international standard book numbers (ISBN) and bar codes
 - 4 Describe how an automatic repeat query (ARQ) can be used to establish that data is received without error
- Including parity byte and parity block check
 - Including the use of:
 - Positive/negative acknowledgements
 - Timeout
 - Asymmetric encryption includes the use of public and private keys

2.3 Encryption

Understand the need for and purpose of encryption when transmitting data.

Understand how data is encrypted using symmetric and asymmetric encryption.

Sami Khatri



+92-335-8467840



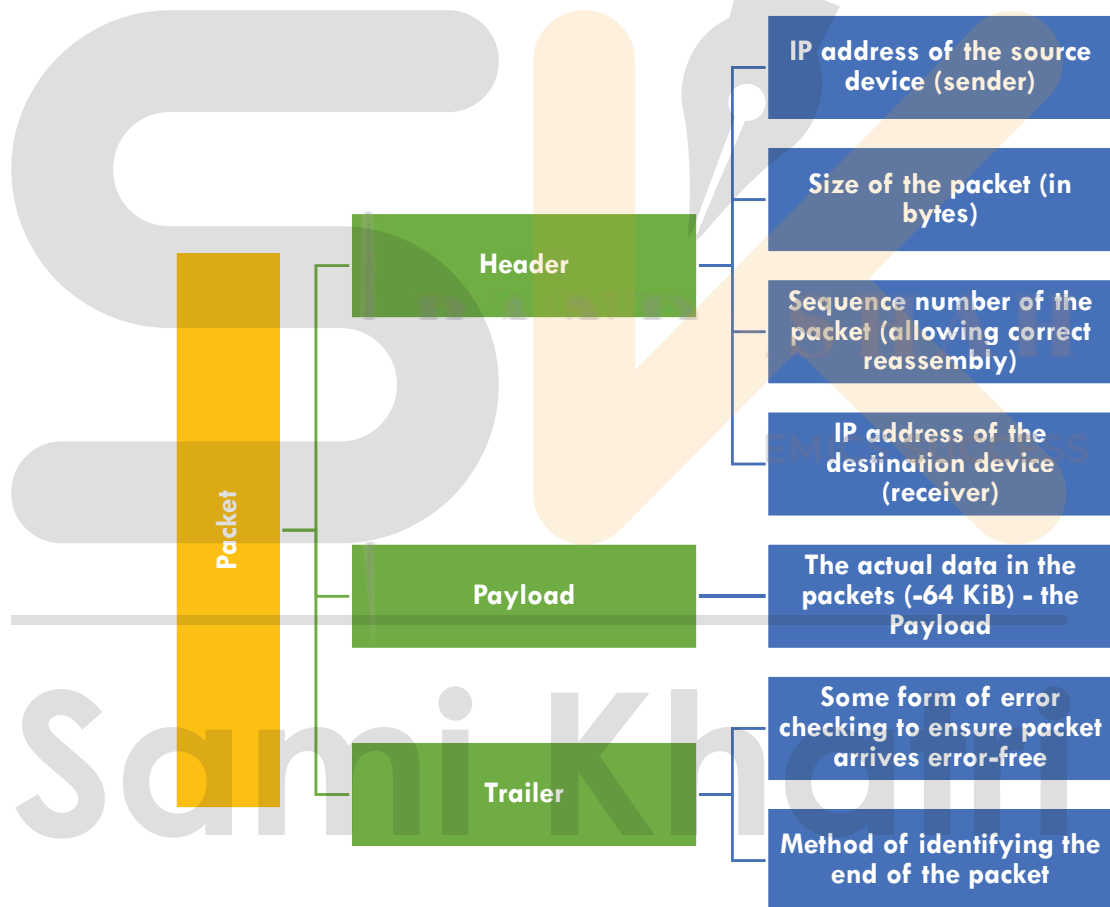
cswithsk@gmail.com

2.1 Types & Methods of Data Transmission

Data Packets

Data sent over long distances is usually broken up into data packets (datagrams). The packets of data are usually quite small, typically 64 KiB, which are much easier to control than a long continuous stream of data. The idea of splitting up data in this way means each packet can be sent along a different route to its destination. This would clearly be of great benefit if a particular transmission route was out of action or very busy. Drawback of splitting data into packets is the need to reassemble the data when it reaches its destination.

Packet Structure



For each packet, the packet header consists of:

- IP address of the sending device
- IP address of the receiving device
- Sequence number of the packet
- Packet size



+92-335-8467840



cswithsk@gmail.com

Payload consists of the actual data being sent in the Packet.

Packet trailer consists of:

- Way of identifying the end of the packet; this is essential to allow each packet to be separated from each other as they travel from sending to receiving station
- An error checking method; cyclic redundancy checks (CRCs) are used to check data packets.

Packet Switching

Method of Data Transmission in which data is split into multiple packets and then each packet to the destination. Upon reaching destination they are reassembled into one in correct order. At each stage in the transmission, there are nodes that contain a router. Each router will determine which route the packet needs to take, in order to reach its destination.

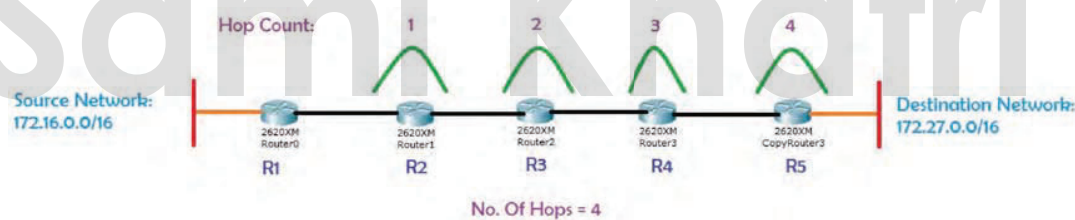
Pros

- No need to tie up a single communication line
- It is possible to overcome failed, busy or faulty lines by simply re-routing packets
- It is relatively easy to expand package usage
- High data transmission rate is possible

Cons

- Packets can be lost and need to be re-sent
- Method is more prone to errors with real-time streaming
- Delay at the destination whilst the packets are being re-ordered

Packets to get lost because they keep bouncing around from router to router and never reach their destination. Eventually the network would just grind to a halt as the number of lost packets mount up, clogging up the system. To overcome this, a method called hopping is used. A hop number is added to the header of each packet, and this number is reduced by 1 every time it leaves a router.



Each packet has a maximum hop number to start with. Once a hop number reaches zero, and the packet hasn't reached its destination, then the packet is deleted when it reaches the next router. The missing packets will then be flagged by the receiving computer and a request to re-send these packets will be made.



+92-335-8467840



cswithsk@gmail.com

Data Transmission

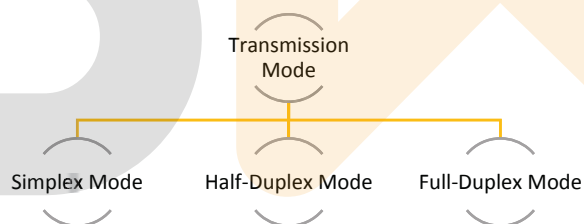
When data is sent from one device to another, it is important to consider how that data is transmitted. It is also important to ensure that the data hasn't been changed in any way. Data transmission can be either over a short distance (for example, from computer to printer) or over longer distances (for example, over a telephone network). Essentially, three factors need to be considered when transmitting data (each factor has to be agreed by both sender and receiver for this to work without error):

- ➡ Direction of data transmission
- ➡ Method of transmission
- ➡ Method of Data synchronization

These factors are usually considered by a communication protocol.

Information flows through the computer in many ways. The CPU is the central point for most information. When you start a program, the CPU instructs the storage device to load the program into RAM. When you create data and print it, the CPU instructs the printer to output the data. Because of the different types of devices that send and receive information, two major types of data transfers take place within a computer: parallel and serial.

Data Transmission Modes

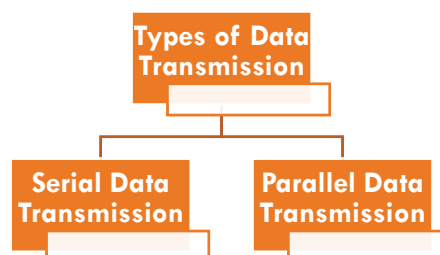


Simplex Data Transmission: Data can be sent in single direction only i.e. sender to receiver e.g. Radio Broadcast.

Half-Duplex Transmission: Data can be sent bidirectional but not at the same time (means data can travel in one direction at a time through same transmission line. e.g. walkie talkie.

Full-Duplex Transmission: Data can be sent bidirectional at the same time through the same transmission line. e.g. Telephone.

Types of Data Transmission



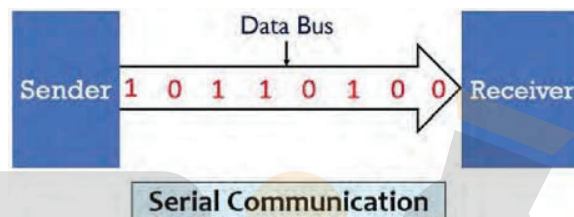
+92-335-8467840



cswithsk@gmail.com

Serial Data Transmission

When data is sent or received using serial data transmission, the data bits are organized in a specific order, since they can only be sent one after another. The order of the data bits is important as it dictates how the transmission is organized when it is received. It is viewed as a reliable data transmission method because a data bit is only sent if the previous data bit has already been received.



Examples of Serial Data Transmission

Telephone Line, Coaxial Cable, Fiberoptic Cable etc.

Serial data transmission can be simplex, half-duplex or full-duplex.

Pros

- Cheap (less number of wires)
- Fast (No syncing)
- Used for Long Distances

Cons

- Slower speed
- Splitting and reassembling creates an overhead

Application

Serial data transfer is mostly used for computer-to-computer transfer or from a computer to an external device located some distance away. Other devices that use serial transfers are:

- USB (Universal Serial Bus) port and devices (USB printers, scanner, webcams etc.)
- Modems
- SATA (Serial ATA) hard disks, CD/DVD drives etc.

Parallel Data Transmission

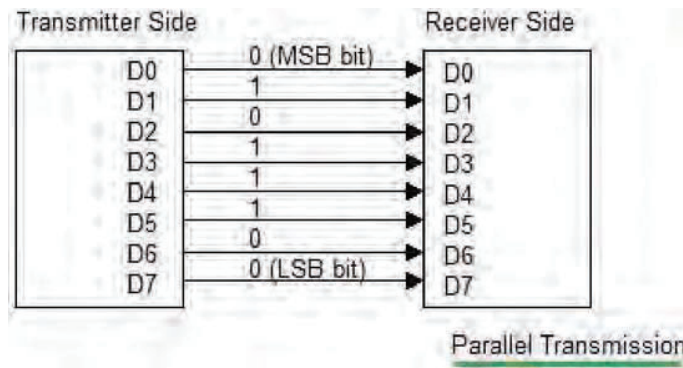
When data is transmitted using parallel data transmission, multiple data bits are transmitted over multiple channels at the same time. This means that data can be sent much faster than using serial transmission methods.



+92-335-8467840



cswithsk@gmail.com



Pros

- Multiple bits are transferred fast over short distances.
- Data does not need to be split and reassembled during transmission which saves time.

Cons

- Expensive (more wires)
- Used for short distances
- Requires syncing i.e. all bits reach destination at the same time or the data is considered corrupted and requires retransmission

Application

Parallel transmission either takes place within a computer system (on a computer bus) or to an external device located a close distance away. It is also used to transfer data between:

- Internal data transfer inside computer (buses)
- CPU and interface cards (i.e. graphics card, sound cards etc.)
- Computer to printer

Serial & Parallel Transmission Comparison

Serial	Parallel
Less risk of external interference than with parallel (due to fewer wires)	Faster rate of data transmission than serial
More reliable transmission over longer distances	Works well over shorter distances (for example, used in internal pathways on computer circuit boards)
Transmitted bits won't have the risk of being skewed (that is, out of synchronization)	Since several channels/wires used to transmit data, the bits can arrive out of synchronization (skewed)
Used if the amount of data being sent is relatively small since transmission rate is slower than parallel	Preferred method when speed is important



+92-335-8467840

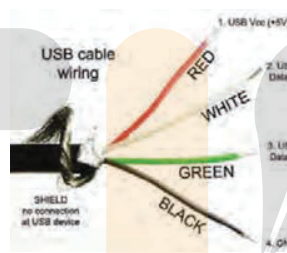


cswithsk@gmail.com

Used to send data over long distances	If data is time-sensitive, parallel is the most appropriate transmission method
Less expensive than parallel due to fewer hardware requirements	Parallel ports require more hardware, making them more expensive to implement than serial ports

Universal Serial Bus (USB)

It is form of Serial Data Transmission. USB allows both half-duplex and full-duplex data transmission.



USB cable consists of a four-wired shielded cable, with two wires for power (red and black). The other two wires (white and green) are for data transmission.

A new type of USB connector, referred to as USB-C, is now becoming more common in laptops and tablets/phones. This is a 24-pin symmetrical connector which means it will fit into a USB-C port either way round.

Sami Khatri



+92-335-8467840



cswithsk@gmail.com

2.2 Methods of Error Detection

Need to Check for Errors

There is a risk that data is corrupted or lost during transmission.

Errors can occur during data transmission due to:

- Interference
- Problems during packet switching
- Skewing of data

Checking for errors is important since computers are unable to understand text, for example, if the words are not recognized by its built-in dictionary.

Number of ways data can be checked for errors in data transmission.

- Parity Checks
- Checksum
- Echo Check

Parity Checks, Checksum & Echo Checks

Parity Checks

One of the methods that can be used to check whether data is corrupted or not. Based on number of '1-bits' in a byte of data.

Parity of two types Even Parity (even number of 1-bits) and Odd Parity (odd number of 1-bits). MSB is reserved for Parity bits. The parity bit is set according to whether the parity being used is even or odd. For example, consider the byte:

	1	0	1	0	1	0	1
--	---	---	---	---	---	---	---



+92-335-8467840



cswithsk@gmail.com

Example: Odd Parity

if the byte is using odd parity, then the parity bit needs to be set to 1, since we need to have an odd number of 1-bits in the byte.

1	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Example: Even Parity

if the byte is using even parity, then the parity bit needs to be set to 0, since there is already an even number of 1-bits in the byte (four 1-bits).

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Sender and Receiver make a agreement before data transmission that which type of parity is being used. It is therefore used as a Parity checking protocol.

An error would be flagged if the byte now had an even number of 1-bits at the receiver's end.

Sent Byte

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Received Byte

0	1	0	0	0	1	0	1
---	---	---	---	---	---	---	---

Now, according to agreement Parity Bit is decided to even parity i.e. even number of 1-bits but the data received contains odd number of bits. This means error occurred during the transmission of the byte. The error is detected by the recipient's computer re-calculating the parity of the byte sent. If even parity had been agreed between sender and receiver, then a change in parity in the received byte indicates that a transmission error has occurred.

Example

In this example, nine bytes of data have been transmitted. Agreement has been made that even parity will be used. Another byte, known as the parity byte, has also been sent. This byte consists entirely of the parity bits produced by the vertical parity check. The parity byte also indicates the end of the block of data.



+92-335-8467840



cswithsk@gmail.com

Table shows how the data arrived at the receiving end. It is now necessary to check the parity of each byte horizontally (bytes 1 to 9) and vertically (columns 1 to 8). Each row and column where the parity has changed from even to odd should be flagged:

	Parity Bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	1	1	1	1	0	1	1	0
Byte 2	1	0	0	1	0	1	0	1
Byte 3	0	1	1	1	1	1	1	0
Byte 4	1	0	0	0	0	0	1	0
Byte 5	0	1	1	0	1	0	0	1
Byte 6	1	0	0	0	1	0	0	0
Byte 7	1	0	1	0	1	1	1	1
Byte 8	0	0	0	1	1	0	1	0
Byte 9	0	0	0	1	0	0	1	0
Parity Byte	1	1	0	1	0	0	0	1

Byte 8 now has incorrect parity (three 1-bits)

Bit 5 (column-5) also now has incorrect parity (five 1-bits)

Table shows that an error has occurred following data transmission.

Intersection of row 8 and column 5, the position of the incorrect bit value (which caused the error) can be found. The 1 -bit at this intersection should be a 0-bit; this means that byte 8 should have been:

0 0 0 1 0 0 1 0

Which would also correct column 5 giving an even vertical parity (now has four 1 -bits).

This byte could therefore be corrected automatically as shown above, or an error message could be relayed back to the sender asking them to re-transmit the block of data.

Checksum

Detection for if data has been changed or corrupted following data transmission. Data is sent in blocks, and additional value, called checksum is sent at the end of the block of data.

Checksum Process

- ➔ Block of data is about to be transmitted, the checksum is calculated from the block of data
- ➔ Calculation is done using an agreed algorithm (this algorithm has been agreed by sender and receiver)
- ➔ Checksum is then transmitted with the block of data
- ➔ At the receiving end, the checksum is recalculated by the computer using the block of data (the agreed algorithm is used to find the checksum)



+92-335-8467840



cswithsk@gmail.com

- Re-calculated checksum is then compared to the checksum sent with the data block
- If the two checksums are the same, then no transmission errors have occurred; otherwise a request is made to re-send the block of data.

Echo Check

Echo Check is a method in which after sending data to other device, data is sent again to the sender to recheck if any error occurred during transmission.

- A copy of the data is sent back to the sender
- Returned data is compared with the original data by the sender's computer
- If there are no differences, then the data was sent without error
- If the two sets of data are different, then an error occurred at some stage during the data transmission.

Check Digits

A check digit is the final digit included in a code; it is calculated from all the other digits in the code. Check digits are used for barcodes on products types can usually detect the following types of error:

- an incorrect digit entered, for example 5327 entered instead of 5307
- transposition errors where two numbers have changed order, for example 5037 instead of 5307
- omitted or extra digits, for example 537 instead of 5307 or 53107 instead of 5307
- phonetic errors, for example 13 (thirteen), instead of 30 (thirty).



+92-335-8467840



cswithsk@gmail.com

There are a number of different methods used to generate a check digit. Two common methods will be considered here:

- ISBN 13
- Modulo-11

ISBN 13

The check digit in ISBN 13 is the thirteenth digit in the number. We will now consider two different calculations. The first calculation is the generation of the check digit. The second calculation is a verification of the check digit (that is, a recalculation).

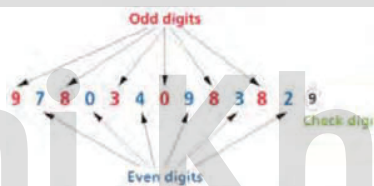
Calculation 1 – Generation of the Check Digit from the other 12 Digits in a Number

- Add all the odd numbered digits together
- Add all the even numbered digits together and multiply the result by 3
- Add the results from 1 and 2 together and divide by 10
- Take the remainder, if it is zero then use this value, otherwise subtract the remainder from 10 to find the check digit.

Using the ISBN 978034098382 (note this is the same ISBN as in Figure):

- $9 + 8 + 3 + 0 + 8 + 8 = 36$
- $3 \times (7 + 0 + 4 + 9 + 3 + 2) = 75$
- $\frac{36+75}{10} = \frac{111}{10} = 11 \text{ remainder } 1$
- $10 - 1 = 9 \text{ (check digit)}$

So, we end up with the following thirteen-digit number



Calculation 2 - Re-calculation of the Check Digit from the thirteen-digit number

To check that an ISBN 13-digit code is correct, including its check digit, a similar process is followed:

- Add all the odd numbered digits together, including the check digit
- Add all the even number of digits together and multiply the result by 3



+92-335-8467840



cswithsk@gmail.com

- ➔ Add the results from 1 and 2 together and divide by 10
- ➔ Number is correct if the remainder is zero. U

Using the ISBN 9780340983829 (including its check digit)

- ➔ $9 + 8 + 3 + 0 + 8 + 8 + 9 = 45$
- ➔ $3 \times (7 + 0 + 4 + 9 + 3 + 2) = 75$
- ➔ $(45 + 75)/10 = 120/10 = 12$ remainder 0
- ➔ Remainder is 0, therefore number is correct.

Modulo-11

The modulo-11 method can have varying lengths of number which makes it suitable for many applications, such as product codes or VINs. The first calculation is the generation of the check digit. The second calculation is a verification of the check digit

Calculation 1 - Generation of the check digit from the other digits in a number

The following algorithm generates the check digit from the other 7 digits:

- ➔ Each digit in the number is given a weighting of 8, 7, 6, 5, 4, 3 or 2 starting from the left (weightings start from 8 since the number will become eight-digit when the check digit is added)
- ➔ The digit is multiplied by its weighting and then each value is added to make a total
- ➔ The total is divided by 11
- ➔ The remainder is then subtracted from 11 to find the check digit (note if the remainder is 10 then the check digit 'X' is used).

The example to be used has the following seven-digit number:

- ➔ 7-digit number: 4 1 5 6 7 1 0
- ➔ weighting values: 8 7 6 5 4 3 2
- ➔ Sum: $(8 \times 4) + (7 \times 1) + (6 \times 5) + (5 \times 6) + (4 \times 7) + (3 \times 1) + (2 \times 0)$
 $= 32 + 7 + 30 + 30 + 28 + 3 + 0$
total = 130
- ➔ Divide total by 11: $130/11 = 11$ remainder 9
- ➔ Subtract remainder from 11: $11 - 9 = 2$ (check digit)

So, we end up with the following eight-digit: 4 1 5 6 7 1 0 2

Calculation 2 - Re-calculation of the check digit from the eight-digit number (which now includes the check digit)

To check that the eight-digit number is correct, including its check digit, a similar process is followed:

- ➔ Each digit in the number is given a weighting of 8, 7, 6, 5, 4, 3, 2 or 1 starting from the left



+92-335-8467840



cswithsk@gmail.com

- The digit is multiplied by its weighting and then each value is added to make a total
- The total is divided by 11
- The number is correct if the remainder is zero

Using the 8-digit number: 4 1 5 6 7 1 0 2

- Weighting values: 8 7 6 5 4 3 2 1
- Sum: $(8 \times 4) + (7 \times 1) + (6 \times 5) + (5 \times 6) + (4 \times 7) + (3 \times 1) + (2 \times 0) + (1 \times 2)$
 $= 32 + 7 + 30 + 30 + 28 + 3 + 0 + 2$
total = 132
- Divide total by 11: $132/11 = 12$ remainder 0
- Remainder is 0, therefore number is correct

Automatic Repeat Requests (ARQ's)

Automatic repeat request (ARQ) is a set of rules for error control when transmitting data. When the device receiving the data detects there is an error with a packet, it automatically sends a request to the device transmitting the data to resend the packet. This resend request will be sent repeatedly until the packet is received error free or a limited amount of resend requests is reached.

Sami Khatri



+92-335-8467840



cswithsk@gmail.com

2.3 Symmetric & Asymmetric Encryption

Purpose of Encryption

When data is transmitted through wired or wireless channel there is always a threat of data theft or hack, so encryption is a method to protect it to a certain extent.

Through encryption data is altered into a certain form i.e. understandable for unauthorized person. It cannot prevent data interception, but it stops it from making the sense.

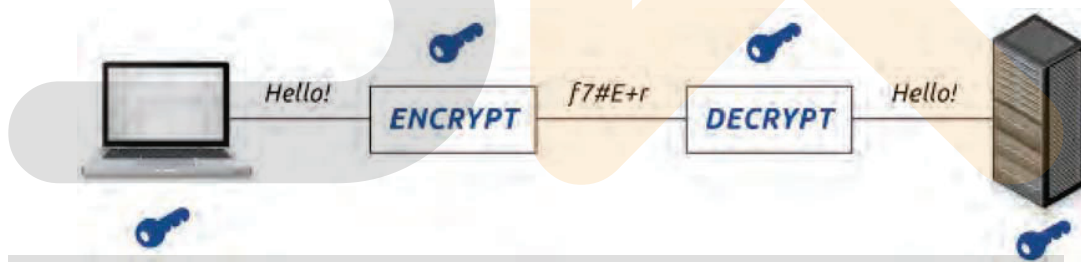
Plaintext and Cypher

Original Data: Plaintext

Encrypted Data: Cyphertext

Symmetric Encryption

Symmetric encryption uses an encryption key, the same key is used to encrypt and decrypt the encoded message.



Asymmetric Encryption

Asymmetric encryption was developed to overcome the security problems associated with symmetric encryption. It makes use of two keys called public key and private key. Public key (made available to everybody) and private key (only known to the computer user). Both types of key are needed to encrypt and decrypt messages.



+92-335-8467840



cswithsk@gmail.com

How Asymmetric Encryption Works



The Binary Number System is the fundamental building component of every computer. Because this system only contains 1's and 0's, it was chosen. Computers may be represented using the binary system because they have countless millions of tiny "switches" that must be in the ON or OFF position. When a switch is in the ON position, it is represented by 1, and when it is in the OFF position, by 0.

Sami Khatri



+92-335-8467840



cswithsk@gmail.com